

Answer the Questions

Question 1: Why is it better to code against an interface rather than a concrete class?

- Coding against an **interface** is often better than coding directly against a **concrete class** because it makes your code **more flexible, maintainable, and easier to test**.

Question 2: When should you prefer an abstract class over an interface?

- You should prefer an **abstract class** over an interface when you have a **strong “is-a” relationship** between classes and you want to share **common code or state** between them.

Interface	Abstract Class
Defines a contract/capability	Defines a common base
Usually focuses on what a class can do	Can define what + how
No instance state	Can have fields/state
No traditional constructors	Can have constructors
A class can implement multiple interfaces	A class can inherit only one class

Question 3: How does implementing IComparable improve flexibility in sorting?

- Implementing **IComparable<T>** improves sorting flexibility because it allows a class or struct to define its **default/natural ordering**. Then standard sorting methods such as `List.Sort()` can sort objects automatically.

Question 4: What is the primary purpose of a copy constructor in C#?

- The **primary purpose of a copy constructor** in C# is to **create a new object by copying the values from an existing object of the same class**.

Why use it?

It is useful when you want to:

- Create a duplicate of an existing object.
- Initialize a new object using another object's data.
- Control whether the copy is **shallow or deep** when the object contains reference types.

Question 5: How does explicit interface implementation help in resolving naming conflicts?

- **Explicit interface implementation** helps resolve naming conflicts when a class implements **two or more interfaces that have methods with the same name**.

Question 6: What is the key difference between encapsulation in structs and classes?

- The key difference is that **encapsulation works similarly in both structs and classes**, but they differ mainly in **how the data is stored and copied**, not in the access modifiers available.

Question 7: what is abstraction as a guideline, what's its relation with encapsulation?

Abstraction as a guideline means:

Focus on what an object does, rather than how it does it.

It tells us to hide unnecessary implementation details and expose only the important parts that the user of the class needs.

Question 8: How does constructor overloading improve class usability?

- **Constructor overloading** improves class usability by allowing you to create objects in **different ways**, depending on the information you currently have.